

Take-Home Assignment: Remote Support Portal (Next.js)

Context & Story

You are building a proof-of-concept for a Remote Field Technician Support Portal. When field technicians arrive at a complex repair site, they use this web application to configure their job details, review safety instructions, and interface with a "Remote Expert System" to diagnose, record, and verify the repair.

We are looking for clean, modular, and reusable code. We care deeply about how you structure your Next.js application, where you draw the line between Client and Server components, how you handle state management across a multi-step workflow, and your architectural decisions regarding data flow.

Tooling & AI Assistants: We embrace modern development workflows. You are fully welcome (and encouraged) to use agentic coding assistants like Google Antigravity, GitHub Copilot, Codex, Claude Code, or Cursor to build this application. We evaluate your architectural decisions, component modularity, and final product quality - not whether you manually typed every character yourself. However, AI is an amplifier, not a crutch. We expect you to deeply understand the code you submit and be fully capable of stepping in to make precise, manual modifications without relying on an agent for every small change or bug fix.

Feature Requirements & User Flow

Phase 1: Job Configuration (Home Page)

- Display a small grid of configuration cards.
- Users must select parameters for their job. Examples:
 - Equipment Type: (e.g., HVAC, Industrial Printer, Server Rack)
 - Severity Level: (e.g., Routine Maintenance, Critical Fault)
- Upon selection, the user clicks "Start Mission", and these parameters are passed to the next page.

Phase 2: Pre-Deployment Briefing (Prep Page)

- Display the parameters selected in Phase 1.
- Display mock safety instructions based on those parameters.
- **Timer functionality:** A countdown timer of 30 seconds must run automatically on this page.
- When the timer reaches 0, the application must automatically redirect the user to the next page (Phase 3).

- Include a "Skip & Proceed" button for users who finish reading before the timer ends.
- Request webcam/microphone permissions on this page (in preparation for the recording step in Phase 3). Handle a denied or unavailable permission gracefully — show a clear inline message and a way to retry — rather than letting the user proceed to Phase 3 unaware their camera won't work.

Phase 3: The Support Workspace (Activity Page)

This is the core of the application. It consists of a single page divided into three distinct vertical sections (Tabs). The user must complete them sequentially; clicking a "Completed - Next" button locks the current tab and highlights/activates the next one.

Global Timer functionality: A countdown timer of 10 minutes (600 seconds) must run automatically on this page. When the timer reaches 0, the application must automatically redirect the user to the next page by gracefully winding up, saving all data and progress done by user.

Tab 1: Initial Assessment (Scoping)

- **Requirement:** Simulate a real-time connection with a "Remote Expert".
- **Implementation:** Build a mock interface simulating a WebSocket or WebRTC data channel. The "Expert" (a hardcoded script) should output an initial greeting and ask a scoping question based on the Phase 1 configuration. The user should be able to type clarifying questions, and the mock system should reply with basic, delayed responses.
- Once the scoping chat is done, the user clicks "Completed - Next".

Tab 2: Repair Documentation (Solution)

- **Requirement:** The technician must record themselves performing the fix.
- **Implementation:** Integrate the browser's MediaRecorder API to capture video/audio from the user's webcam. Provide Start, & Stop ("Completed – Next") controls.
- Once the video is recorded and confirmed, the user clicks "Completed - Next".

Tab 3: Quality Assurance (Q&A)

- **Requirement:** A final follow-up chat with the "Remote Expert" after the video is submitted.
- **Implementation:** Similar to Tab 1, mock a real-time messaging interface where the system asks 1-2 generic follow-up questions, and the user replies.
- Clicking "Finish Job" ends the flow and routes the user to a placeholder "Performance Analysis" screen (no need to design the final screen, just create a dummy and route to it).

Technical & Architecture Requirements

1. **Next.js Paradigms (App Router):** You must demonstrate a strong understanding of modern Next.js architecture. We will heavily evaluate your deliberate division of Server Components vs. Client Components. Furthermore, your code must showcase

a conscious choice between using Server Actions (e.g., for UI-driven data mutations and state updates) versus Route Handlers (e.g., for RESTful API boundaries or handling external webhooks).

2. **Route Protection:** A user should not be able to jump directly to /activity (or to Tab 3) by typing the URL, without having completed the prior steps. Handle this however you think is cleanest — middleware, a layout-level check, or a server-side guard.
3. **Browser-Only APIs:** MediaRecorder and any mock WebSocket/WebRTC channel only work in the browser. Make sure these are correctly isolated in Client Components and, where it helps avoid unnecessary server-side evaluation. We're specifically looking at whether you understand *why* this isolation is needed, not just that it works.
4. **Error Handling:** Handle at least one real failure case gracefully — specifically, the user denying webcam/microphone permissions on the Prep Page. Use an error.tsx / error boundary or inline recovery UI; don't let it hard-crash the page, and don't let the user reach Phase 3 without knowing their camera access failed.
5. **Backend Architecture (Optional but Encouraged):** While Next.js can act as a full-stack framework, you may choose to build a separate backend server/codebase (e.g., Node/Express, Python). If you take this route, explain your architectural rationale in the README.md. Context: In a production environment, separating the backend is often necessary for managing persistent WebSocket connections (for the real-time chat in Tabs 1 & 3) or handling heavy, long-running tasks like processing and storing video blobs without tying up the Next.js server. Only necessary operations are to be separated into a separate backend.
6. **Data Persistence:** You must store the user's progress and inputs to mimic a database. Use local storage, a mock JSON server, or your backend/API layer. Think carefully about when to save data (e.g., on keystroke, on tab completion, or at the end of the flow) to balance UI responsiveness with data integrity.
7. **Graceful Loading & UX:** Field technicians often deal with poor network conditions. You must implement thoughtful loading states. Use Next.js features like loading.tsx and React Suspense boundaries, alongside skeleton loaders or spinners, during route transitions, mock API calls, and video processing.
8. **Component Architecture & Modularity:** Complex UI segments, such as the multi-tab interface, must be highly modular and maintainable. If implementing tabs via client-side state, you must strictly avoid monolithic files by isolating each tab's content into its own dedicated component. Furthermore, you are expected to demonstrate performance optimization by utilizing Next.js dynamic imports (next/dynamic) to lazy-load heavy tab components only when they are actively requested by the user.
9. **Reference Material:** For architectural decisions regarding server/client boundaries and API handling, you may refer to this discussion as a baseline:
<https://github.com/orgs/community/discussions/190342>.
10. **Bonus: Real AI Integration:** The baseline assignment asks you to mock the "Remote Expert" system. However, as an optional bonus, you can wire the expert up to a live LLM instead of using a hardcoded script. You can implement this easily

using the Vercel AI SDK paired with a free API provider (such as Google AI Studio for Gemini or Groq—no credit card required) to dynamically handle the scoping and Q&A tabs. This is strictly optional.

What We Are Evaluating

- **Paradigm Application:** Did you use Server Actions where they make sense, or did you unnecessarily rely on Route Handlers for everything? Is your client bundle kept small by maximizing Server Components?
- **Modularity & Code Structure:** Are your UI components (Cards, Timers, Chat bubbles, Video player) reusable and decoupled from business logic? Is the repository well-organized so another developer can easily navigate the API layer and UI?
- **State Management:** How cleanly is state passed between Phase 1, 2, and 3? How do you manage the active/locked states of the tabs?
- **Robustness:** Does the app handle browser-only APIs, permission failures, and direct/out-of-order URL navigation gracefully?
- **System Design & Separation of Concerns:** If a separate backend was built, is the boundary between the Next.js frontend and the external service logical?
- **UX Polish:** Do your loading states actively prevent the user from feeling lost or double-clicking buttons during simulated network delays?

Submission

Please provide a link to a public GitHub repository. Include a README.md with instructions on how to run the app locally, and a brief paragraph explaining your architecture choices, specifically detailing how you handled data persistence, component rendering (Client vs. Server), and why you chose to use (or not use) a separate backend.

Appendix

Mocking the Voice-First AI & Real-Time Communications

Because you do not have access to live AI endpoints (LLMs, STT, or TTS) for this take-home assignment, you are expected to simulate a "Voice-First" AI workflow using the provided JSON mock data.

How to Implement the Simulation:

1. **The WebRTC / WebSocket Illusion:**
 - Do not simply fetch the entire JSON array and render it instantly.

- You must build a service, hook (e.g., `useMockExpertConnection`), or class that mimics a real-time data channel. The system can "emit" the expert's messages with a deliberate delay (e.g., 1.5 to 3 seconds) to simulate network latency and AI processing time.

2. Simulating STT (Speech-to-Text):

- The camera feed in Tab 2 and the microphone access should be *functional* (e.g., successfully requesting browser permissions and displaying a local media stream), but we do not expect you to wire up actual live transcription APIs.
- To simulate STT in Tabs 1 and 3, when it is the user's turn to "speak," you can either:
 - Provide a "Simulate Speech" button that automatically pulls the next "sender": "user" string from the JSON script and displays it in the chat as if it were just transcribed.
 - Allow the user to type the exact response from the JSON into an input box to trigger the next expert step.

3. Simulating TTS (Text-to-Speech):

- When the mock WebSocket "receives" a message from the "sender": "expert", it must render into the UI chat log.
- **Bonus/Encouraged:** Use the browser's native `window.speechSynthesis` API to literally speak the expert's text response out loud when it arrives, completing the Voice-First illusion.

4. State Management:

- Your application must keep track of the current "step" in the conversation. If the user navigates away from the tab or the network drops, how does your mock system know whether to repeat the last question or wait for the user's response?

UI Mockups & Layout Expectations

To help you visualize the expected flow, we have provided a set of reference mock screens:

Key Layouts to Implement:

1. **Global Layout:** A consistent top Navigation Bar across all active routes.
2. **Phase 1 (Home):** A clear, selectable grid or card system for job parameters.

3. **Phase 2 (Prep):** A prominent countdown timer and a clean display of dynamic safety parameters.
4. **Phase 3 (Workspace):** A split-screen dashboard where the left panel is reserved for technician input (Webcam/Text) and the right panel acts as the persistent, real-time "Remote Expert" AI interface.

SupportPortal

HomeMy JobsDashboard

Job Configuration

Equipment Type

HVAC

Server Rack

Industrial Printer

Severity Level

Routine Maintenance

Critical Fault

Start Mission

SupportPortal

HomeMy JobsDashboard

Pre-Deployment Briefing

1 Configure
Select job parameters.

2 Prepare
Review safety & setup.

3 Execute
Connect with Remote Expert.

Preparation Time: 00:30

Your current job parameters — **Critical Fault** on a **Server Rack**

Hardware Setup Required

The workspace requires access to your camera and microphone for real-time AI analysis.

Grant Media Permissions

Safety Instructions (Mandatory)

1. Ensure anti-static wristband is secured.

2. Verify power redundancy before touching the main board.

3. Do not proceed if the rack temperature exceeds 40°C.

Acknowledge & Proceed

